

MATEMÁTICAS DISCRETAS

Teoría de Árboles

UNIVERSIDAD NACIONAL DE LOJA

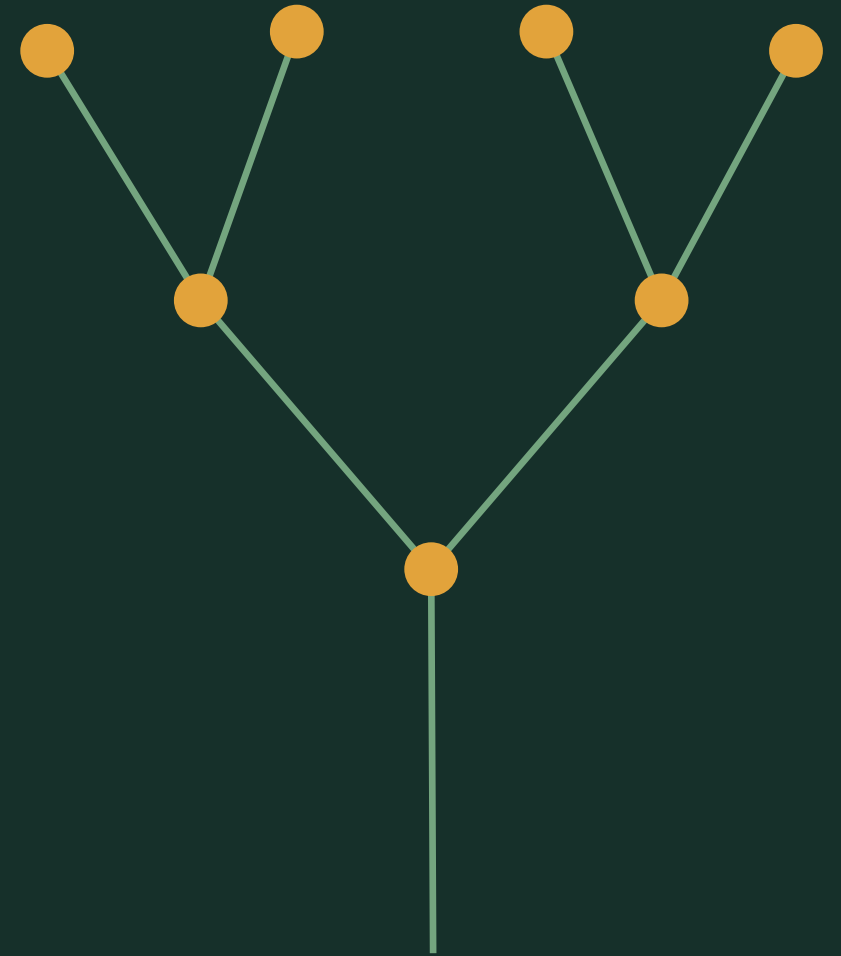
CARRERA DE COMPUTACION

GRUPO C

Integrates: Andy Ordoñez , Miguel Rojas, David Luna , Karen Lalangui

Docente: Ing. Mario Cueva

14 / 07 / 2026



Agenda

01

Tipos de árboles: definiciones y aplicaciones

02

Árboles enraizados y longitud de paseo

03

Códigos de prefijo

04

Árboles de Búsqueda Binaria (ABB)

05

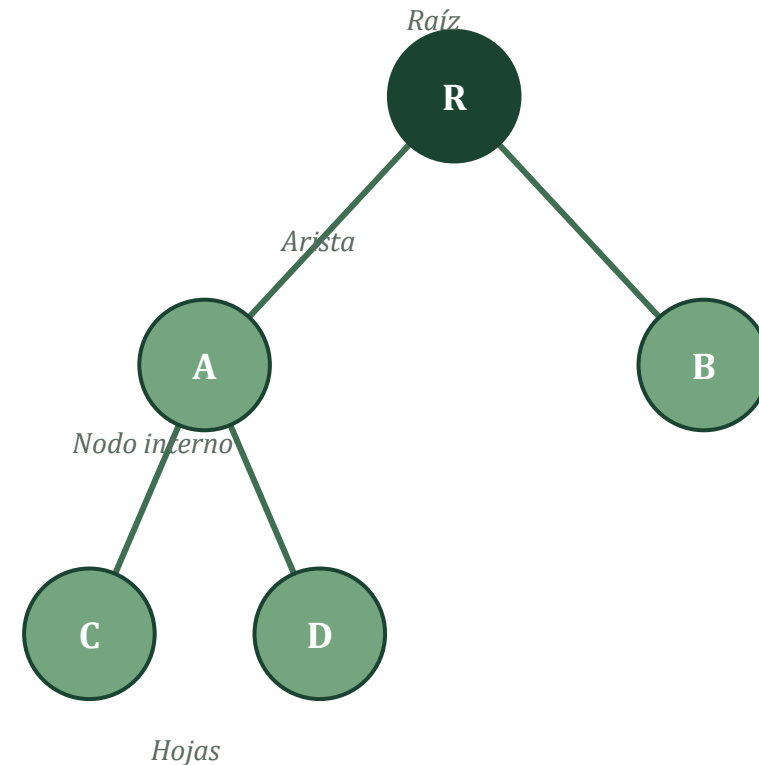
Recorridos: Preorden, Inorden y Postorden

06

Ejemplos prácticos y conclusiones

Definiciones de Árboles y Aplicaciones

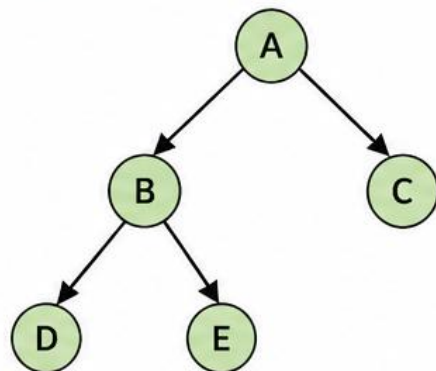
- Un árbol es un grafo no dirigido, conexo y sin circuitos: existe un único camino entre cada par de vértices.
- Componentes clave: raíz, nodo, arista, nodo hoja y subárbol.
- Árbol enraizado: cada nodo (salvo la raíz) tiene exactamente un padre.
- Aplicaciones: sistemas de archivos, algoritmos de búsqueda, códigos de prefijo (Huffman) y modelado de decisiones.



Un grafo se considera árbol si es no dirigido, conexo y no contiene circuitos.

Tipos de Árboles

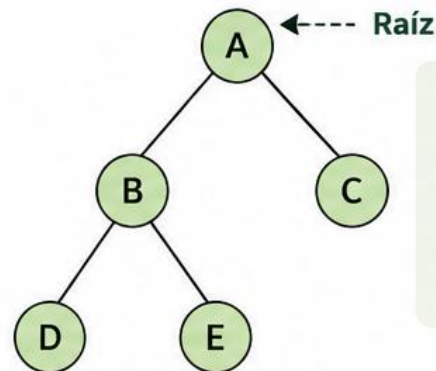
1. ÁRBOL DIRIGIDO



Las aristas tienen una dirección (→).

Ejemplo: Flujo de información o dependencias entre procesos.

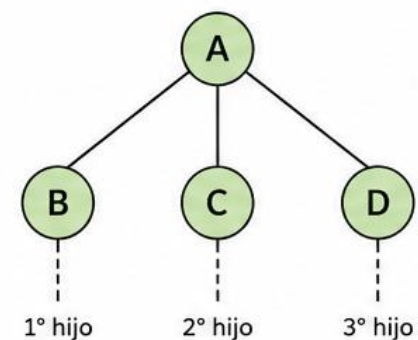
2. ÁRBOL ENRAIZADO



Tiene un único nodo raíz; cada nodo (salvo la raíz) tiene exactamente un padre.

Ejemplo: Sistema de archivos, árbol genealógico, DOM de HTML.

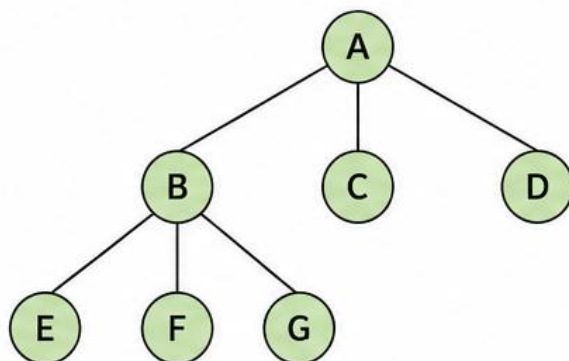
3. ÁRBOL ORDENADO



Los hijos de cada nodo tienen un orden específico.

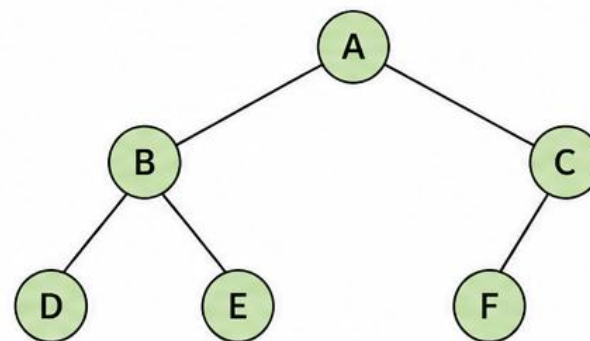
Ejemplo: Expresiones matemáticas, compiladores.

4. ÁRBOL m-ARIO ($m = 3$)



Cada nodo puede tener como máximo m hijos. Aquí $m = 3$.

5. ÁRBOL BINARIO ($m = 2$)

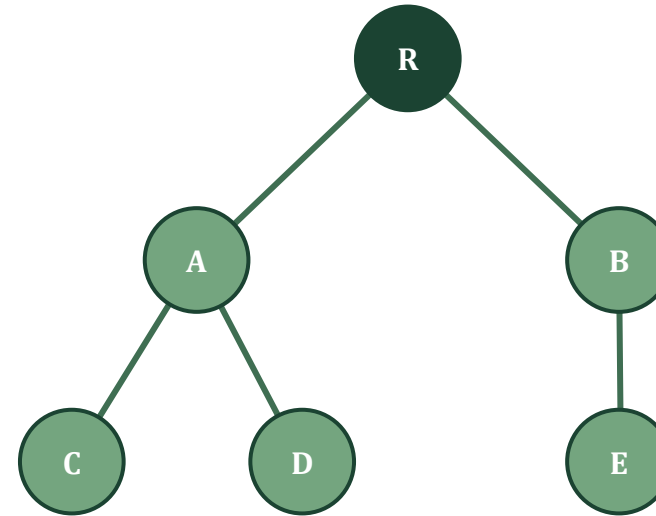


— Hijo izquierdo
— Hijo derecho

Cada nodo puede tener como máximo dos hijos: izquierdo y derecho.

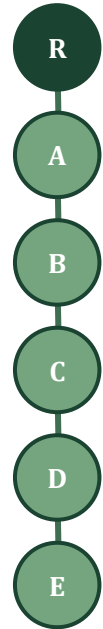
Árboles Enraizados y Longitud de Paseo

- Árbol enraizado: jerarquía estricta de vértices a partir de un nodo raíz único.
- Longitud de paseo: suma de las distancias (en aristas) desde la raíz hasta cada vértice.
- A mayor altura, mayor longitud de paseo y mayor costo de acceso a los datos.
- Un árbol balanceado logra altura $O(\log n)$; uno degenerado cae en $O(n)$.



Balanceado

$$L = 0 + 1 + 1 + 2 + 2 + 2 = 8$$



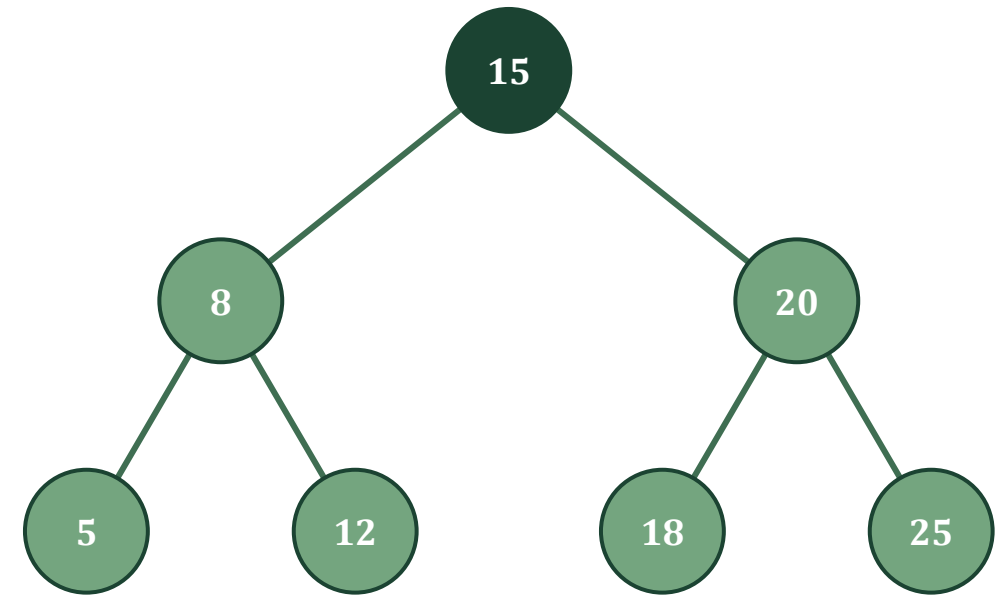
Degenerado

$$L = 1 + 2 + 3 + 4 + 5 = 15$$

El balanceo reduce casi a la mitad el costo total de acceso a los datos.

Árboles de Búsqueda Binaria (ABB)

- Regla recursiva: en cada nodo, el subárbol izquierdo es menor y el derecho es mayor que la clave del nodo.
- Búsqueda: se compara con la raíz y se desciende a la izquierda o derecha según corresponda.
- Inserción: siempre ocurre en la posición vacía donde termina la búsqueda (nodo hoja).
- Eficiencia: $O(\log n)$ si está balanceado; se degrada a $O(n)$ si el árbol es lineal.

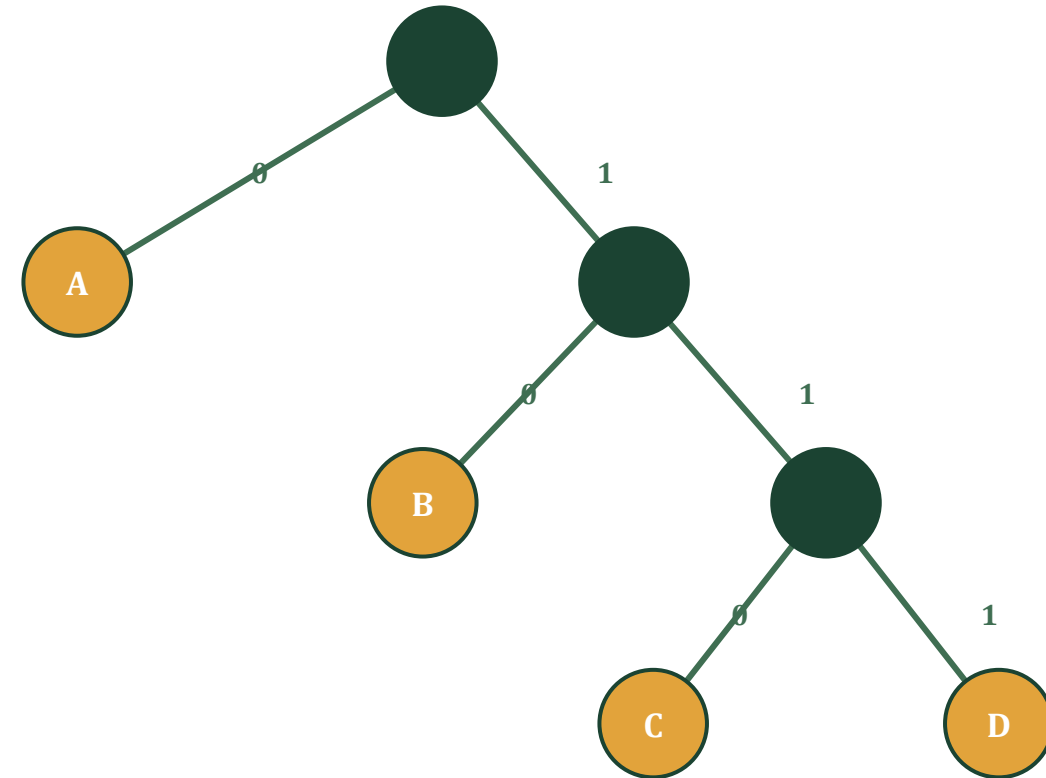


Secuencia de inserción: 15, 8, 20, 5, 12, 18, 25

Buscar 12: $12 < 15 \rightarrow \text{izq.}$ · $12 > 8 \rightarrow \text{der.}$ · $12 = 12 \rightarrow$ ¡encontrado en 3 pasos!
Eliminar 12 (hoja): se desconecta y se limpia el puntero derecho de 8.

Códigos de Prefijo

- Cada símbolo se ubica en una hoja; el camino raíz→hoja define su código binario.
- Propiedad "libre de prefijo": ningún código es el inicio de otro código.
- Símbolos frecuentes cerca de la raíz → menos bits, mayor eficiencia de transmisión.
- Se decodifica sin ambigüedad, bit a bit, sin necesitar separadores.

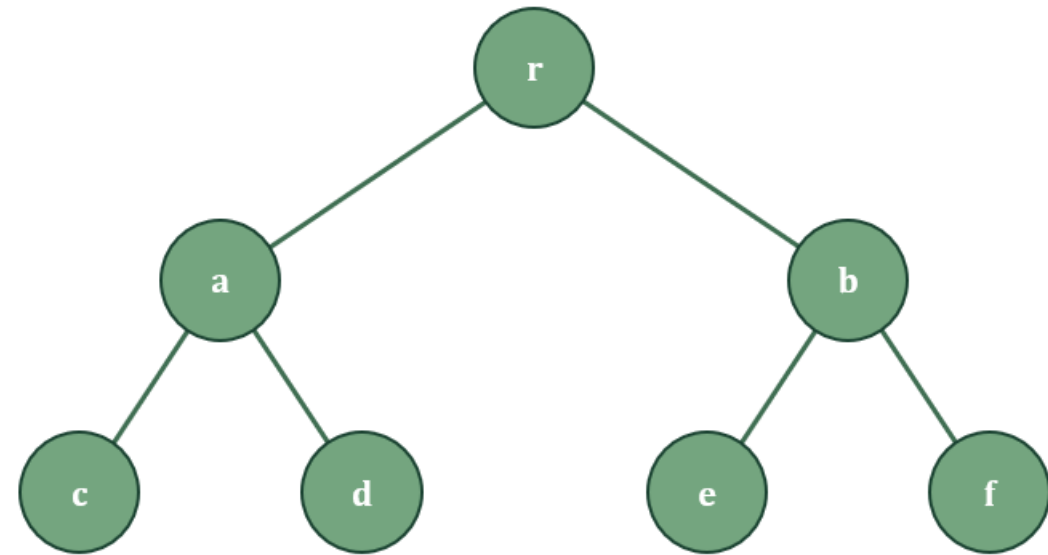


A=0 (50%) B=10 (30%) C=110 (15%) D=111 (5%)

"AABCA" → 0+0+10+110+0 = 00101100

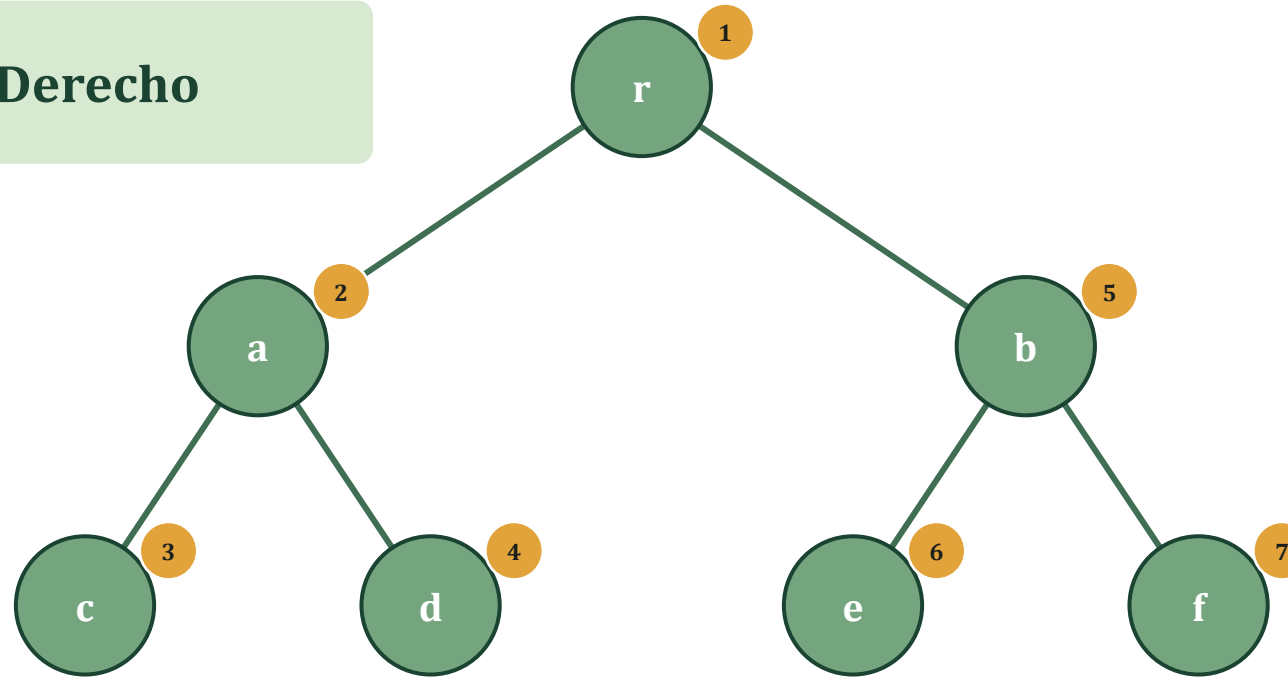
Recorridos en un Árbol: Árbol de Ejemplo

- Un recorrido visita cada nodo una única vez, en un orden definido.
- Convención: N = Nodo raíz, I = subárbol Izquierdo, D = subárbol Derecho.
- Usaremos este árbol de 7 nodos para ilustrar Preorden, Inorden y Postorden.
- Raíz r; subárbol izquierdo encabezado por a (hijos c, d); subárbol derecho encabezado por b (hijos e, f).



Preorden (N – I – D)

Nodo → Izquierdo → Derecho

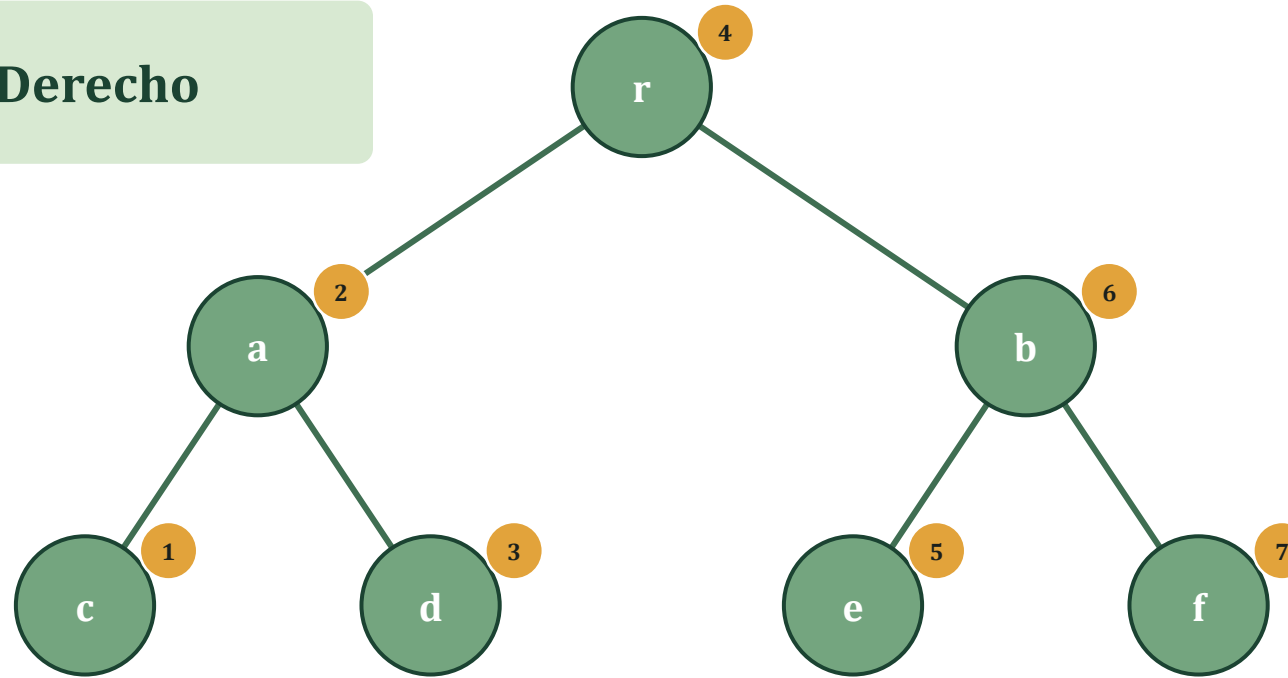


- Paso 1 (N): se visita la raíz r.
- Paso 2 (I): se aplica NID al subárbol de a → a, c, d.
- Paso 3 (D): se aplica NID al subárbol de b → b, e, f.
- Secuencia final: r - a - c - d - b - e - f



Inorden (I – N – D)

Izquierdo → Nodo → Derecho

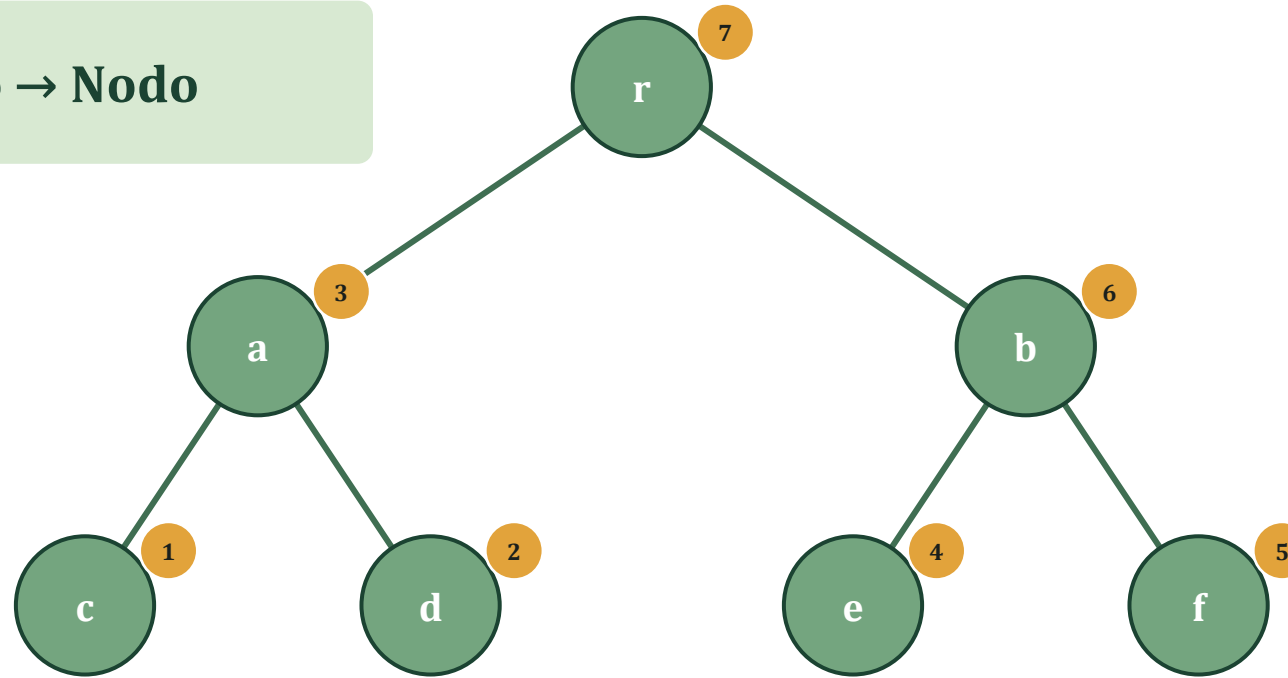


- Paso 1 (I): se desciende hasta el nodo más a la izquierda, c.
- Paso 2 (N-D): se visita a, luego su derecho d.
- Paso 3 (N): se visita la raíz principal r.
- Paso 4 (I-N-D): en el subárbol derecho, e - b - f.



Postorden (I – D – N)

Izquierdo → Derecho → Nodo

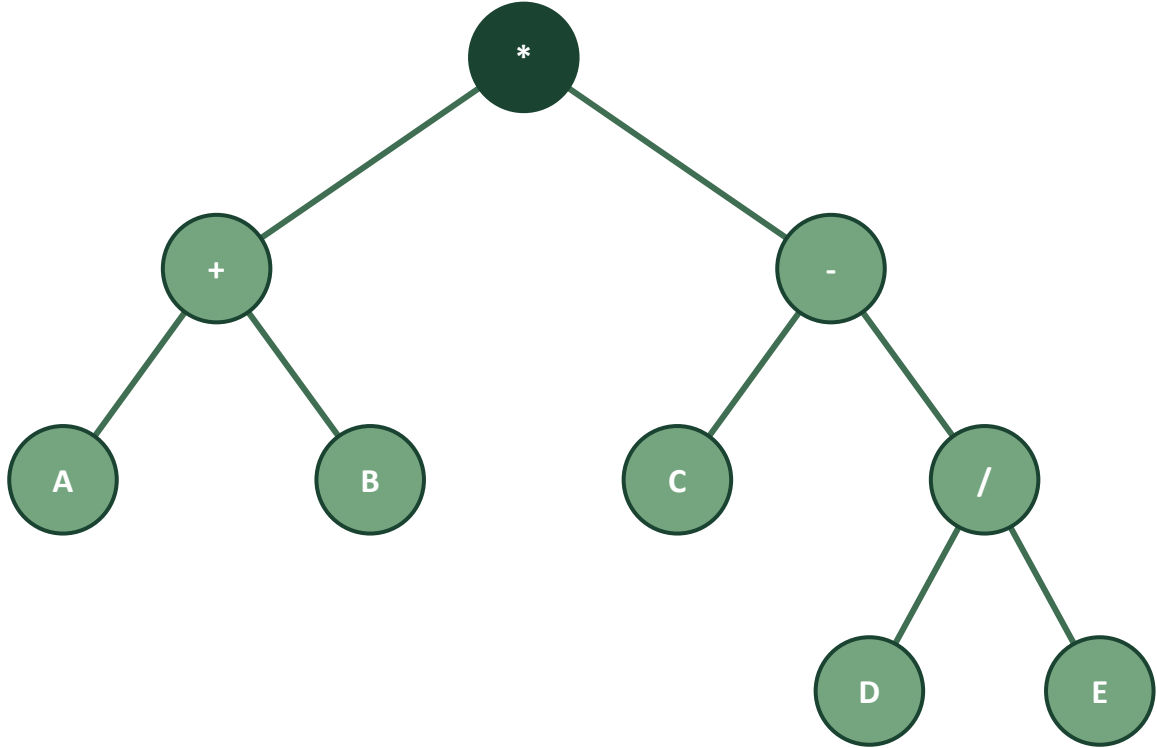


- Paso 1 (I): subárbol izquierdo → c, d, y su raíz local a.
- Paso 2 (D): subárbol derecho → e, f, y su raíz local b.
- Paso 3 (N): finalmente, la raíz principal r.
- Secuencia final: c - d - a - e - f - b - r



Ejemplos Prácticos de Recorridos

Ejercicio: árbol de expresión $(A + B) \times (C - (D / E))$



Preorden: * + A B - C / D E

Inorden: A + B * C - D / E

Postorden: A B + C D E / - *

Otros ejercicios resueltos en la Fase 3

2 **Árbol desbalanceado a la izquierda**
Postorden: 6-4-7-5-2-3-1

3 **BST alfabetizado (raíz M)**
Inorden entrega A-Z ordenado

4 **Árbol “Zig-Zag”**
Inorden = Postorden: 40-30-20-10

5 **Árbol estrictamente binario (raíz 50)**
Cada nodo tiene 0 o 2 hijos

Conclusiones

1

La estructura determina la velocidad

Un árbol balanceado (AVL) preserva $O(\log n)$; un árbol degenerado degrada a $O(n)$.

2

La optimización de rutas es esencial

Reducir la longitud de paseo minimiza el costo de búsqueda, inserción y recuperación.

3

Los códigos libres de prefijo garantizan integridad

Codificar en las hojas elimina ambigüedad en la transmisión de datos.

4

Los recorridos habilitan el acceso sistemático

Preorden, Inorden y Postorden permiten recuperar la información almacenada en el árbol.

La organización lógica de los árboles es la piedra angular de la ingeniería de software eficiente.